



← Back to student guides

✦ Airflow

Orchestration

3 levels

113 sections



The Complete Guide

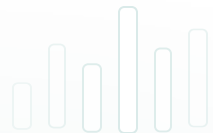
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

DAG

SCHEDULER

EXECUTOR

WORKER



15

sections

14

examples





work you own when Airflow is your responsibility rather than your tool.

Up to this point the questions have been about making a DAG run correctly. From here they are different: who can see and trigger which pipeline, where the credentials live, which component breaks first under load, what the whole thing costs, how you upgrade it without a maintenance window nobody will grant you, and what you do at 3am when the scheduler stops scheduling.

Where this picks up

Topic from earlier levels	What this level adds
The five components	Which one is your bottleneck , and the metrics that tell you before users do
Executors	Cost modelling, idle capacity, and multi-executor routing by workload
Connections and Variables	Secrets backends : credentials never in the metadata database
The webserver and UI	RBAC, per-DAG access control, and what multi-tenancy means
Top-level code	Parse-time budgets as an SLO, and DAG-processor isolation
The metadata database	Growth, <code>db clean</code> , query hot spots, and connection pooling
Pools and concurrency	Fair sharing across teams, and noisy-neighbour containment
Deferrable operators	Triggerer capacity planning and async failure modes
Datasets	Cross-team contracts, and lineage that survives a reorg
SLAs and callbacks	Real SLOs, error budgets, and alert routing that scales
Custom operators	A shared library as a product, with versioning and a release cadence



Testing and CI	Deployment strategy, DAG rollout, and blocking a bad DAG at the gate
Backfills	Backfills that do not take production down
new	Upgrades · observability · incident playbooks · platform ownership · where Airflow ends

I am starting with the security model, because every other decision in this track depends on it.

The security model

Airflow's security surface is larger than people expect, and the reason is a single fact: **a DAG file is arbitrary Python that Airflow executes with the scheduler's and workers' privileges.**

Actor	Can	Should not
DAG author	Write Python that runs on your workers	Read secrets outside their scope
A user with UI access	See DAG code, logs, connections, variables	See another team's credentials in a log
A user who can trigger a DAG	Run arbitrary code the DAG contains	Trigger another team's production pipeline
The scheduler	Execute every DAG file it can see	Anything else
A worker	Run task code with its own credentials	Hold credentials for unrelated systems

Three properties make this sharper than it first appears:

Anyone who can write a DAG file has remote code execution on your cluster. There is no sandbox. A DAG file could read every connection in the metadata database, exfiltrate them,



Logs leak secrets by default. Airflow masks values it knows are sensitive, based on connection fields and a configured list of key names. A credential you fetched yourself and printed is not masked. This is the most common real-world Airflow security incident, and it is entirely preventable.

airflow.cfg

Copy

```
[core]
hide_sensitive_var_conn_fields = True
sensitive_var_conn_names = api_token, private_key, client_secret
```

The UI shows more than people assume. By default an authenticated user can read every DAG's source, every log, and the list of connections. Variables' `values` are masked when the name looks sensitive, which is a heuristic, not a boundary.

Write access to the DAG folder is administrative access

No amount of RBAC configuration limits what a DAG file can do once the scheduler parses it. If contributors must not have cluster-wide privileges, they must not be able to put files in the DAG folder directly. Everything goes through a pull request and a deployment pipeline. Treating the DAG folder as a code artifact rather than a shared drive is the single most important security decision in an Airflow deployment.

► TRY IT

1. List everyone who can write to your DAG folder, including anyone with access to the bucket, the volume, or the git branch it syncs from.
2. In a test environment, write a DAG that reads a connection and prints its password. Confirm whether the log masks it.
3. Now fetch the same secret manually into a variable with a non-obvious name and print it. Check the masking again.
4. Log in as a low-privilege user and see how much DAG code and how many connections you can read.



only user who can see more than intended. Step three is the one that changes how people write logging.

RBAC and multi-tenancy

Airflow ships role-based access control. Understanding what it does and does not give you is the difference between a plan and a hope.

Role	Typical use
Admin	Platform team only
Op	Can manage connections, variables, pools: a senior operator
User	Can trigger and clear DAGs
Viewer	Read-only
Public	Unauthenticated: should have nothing

Per-DAG permissions are the mechanism that makes team separation possible:

BASH

Copy

```
# A role scoped to one team's DAGs
airflow roles create team_billing
airflow roles add-perms team_billing \
  -r can_read -v DAG:billing_ingest \
  -r can_edit -v DAG:billing_ingest
```

PYTHON

Copy



```
dag_id= billing_ingest ,  
access_control={"team_billing": {"can_read", "can_edit"}},  
...  
)
```

✓ RBAC does give you

- Per-DAG read, edit, and delete permissions
- Separation of who can trigger from who can only look
- Restricting connection and variable management
- SSO integration via the auth manager

✗ RBAC does not give you

- Isolation between task code from different teams
- Preventing a DAG from reading another team's connection
- Stopping one team's DAG exhausting shared workers
- Any protection against a malicious DAG author

So genuine multi-tenancy needs infrastructure, not configuration:

Isolation need	Mechanism
Task code separation	<code>KubernetesExecutor</code> with per-team namespaces and service accounts
Credential separation	A secrets backend with per-team paths and IAM scoping
Resource separation	Separate Celery queues or Kubernetes node pools, plus pools
Full separation	Separate Airflow deployments: the honest answer for hostile tenants

One Airflow for mutually untrusting teams is the wrong architecture

If two teams must not be able to read each other's data or credentials, and both can author DAGs, one deployment cannot deliver that, because a DAG file runs arbitrary code. The answer is separate deployments, or a model where nobody authors DAGs directly and pipelines are generated from a restricted specification. Saying this in an interview is a senior signal; claiming RBAC solves it is not.



1. Create a role scoped to a single DAG and a user with only that role. Log in as them.
2. Confirm they cannot see other DAGs, and check whether they can still see the connections list.
3. Write a DAG under that team's ownership that reads a connection belonging to another team. Confirm it works.
4. Write down which of your isolation requirements RBAC satisfies.

Expected result – per-DAG visibility that works, and a task that reads a credential it has no business reading. Step three is the demonstration that separates "we have RBAC" from "we have isolation".

Secrets backends

The strongest fix for the previous section's problem is to take credentials out of Airflow entirely.

airflow.cfg

Copy

```
[secrets]
backend = airflow.providers.hashicorp.secrets.vault.VaultBackend
backend_kwargs = {
    "connections_path": "airflow/connections",
    "variables_path": "airflow/variables",
    "mount_point": "secret",
    "url": "https://vault.internal:8200",
    "auth_type": "kubernetes",
    "kubernetes_role": "airflow"
}
```

Airflow resolves a `conn_id` through the backend chain: the secrets backend first, then environment variables, then the metadata database. So `conn_id="warehouse"` becomes a lookup at `secret/airflow/connections/warehouse` in Vault, with nothing stored locally.



Metadata database	Manual	None, any DAG reads any connection	None
Environment variables	Restart required	Per worker, if you segregate workers	None
Vault / AWS Secrets Manager / GCP Secret Manager	Native, automatic	Yes, by path and IAM	Yes, per read

Three consequences worth stating:

Rotation stops being a project. A credential rotated in Vault is picked up on the next task run with no Airflow change and no restart.

Reads become auditable. The secret store logs which identity read which secret when, which is the evidence an auditor asks for and the metadata database cannot provide.

Per-team scoping becomes real. Workers in the billing namespace authenticate with a service account that can only read `secret/airflow/connections/billing/*`. A DAG that asks for another team's connection gets a denial from Vault rather than a password from your database.

A secrets backend is on the hot path of every task

Every `conn_id` resolution is a network call. Airflow caches within a task, but a DAG with fifty tasks each opening a connection is fifty lookups, and a slow or unavailable secret store means tasks failing everywhere at once. Configure the backend's timeout and retry, monitor its latency as a first-class dependency, and never point it at something with a lower availability target than Airflow itself.

► TRY IT

1. Configure a secrets backend in a test environment. The local filesystem backend is enough to see the mechanism.
2. Move one connection out of the metadata database into it and confirm the DAG still works.
3. Delete the connection from the Airflow UI and confirm the DAG



4. Point the backend at an unreachable address and observe how the task fails.

Expected result – a working pipeline whose credentials do not exist in Airflow, and a clear failure mode when the backend is down. Step three is the moment the chain of resolution becomes concrete.

Scaling: which component breaks first

Mid taught you the components. At scale you need to know **which one saturates**, because the symptom is always "Airflow is slow" and the cause is one of four different things.

SATURATION ORDER. EACH ONE PRESENTS AS "AIRFLOW IS SLOW"

1 • DAG parsing

New DAGs appear slowly; scheduler CPU pinned

2 • Scheduler loop

Tasks sit in `scheduled` before reaching `queued`

3 • Metadata database

Everything slow; UI pages time out. **The usual real ceiling**

4 • Worker capacity

Tasks sit in `queued`



DIAGNOSE BEFORE YOU SCALE

`airflow dags report`

Per-file parse duration



Slow query log

Connection count, write volume

Queue depth

Worker CPU and slots



Fix top-level code

```
min_file_process_interval , .airflowignore
```

More schedulers

Active-active, row-level locking

db clean first

Retention beats a bigger instance

More workers, pools

```
worker_concurrency
```

Read the order literally. Adding workers when the database is the ceiling makes things worse, because more workers means more heartbeats and state writes. Diagnose which lane you are in before changing anything, and note that three of the four fixes are configuration or retention, not more hardware.

Bottleneck	Symptom	Diagnose with	Fix
DAG parsing	New DAGs appear slowly; scheduler CPU pinned	<code>airflow dags report</code> , DAG Processing page	Fix top-level code; more parsing processes; <code>.airflowignore</code>
Scheduler loop	Tasks sit in <code>scheduled</code> before <code>queued</code>	<code>scheduler_heartbeat</code> , loop duration metric	More scheduler replicas; raise <code>max_tis_per_query</code>



Metadata database	Everything slow, UI pages time out	Slow query log, connection count	db clean, indexes, a bigger instance; PgBouncer
Worker capacity	Tasks sit in <code>queued</code>	Queue depth, worker CPU	More workers; more <code>worker_concurrency</code> ; pools

airflow.cfg

Copy

```
[scheduler]
# Parsing
parsing_processes = 4                # parallel DAG file processors
min_file_process_interval = 60       # re-parse each file at most this often
dag_dir_list_interval = 120          # rescan the folder this often
file_parsing_sort_mode = modified_time # prioritise recently changed files

# Loop throughput
max_tis_per_query = 512              # task instances examined per query
scheduler_heartbeat_sec = 5
use_row_level_locking = True         # required for multiple schedulers

[database]
sql_alchemy_pool_size = 10
sql_alchemy_max_overflow = 20
sql_alchemy_pool_recycle = 1800
```

Three scaling facts that matter more than the rest:

Multiple schedulers are active-active. Since Airflow 2 you run two or three schedulers behind row-level locking and they share the work. This is the answer to both throughput and availability, and it needs a database that supports `SELECT ... FOR UPDATE SKIP LOCKED`: Postgres 12+ or MySQL 8+.

The metadata database is almost always the real ceiling. Every heartbeat, state change, XCom write, and UI page is a query. A cluster running fifty thousand task instances a day generates a large amount of write traffic, and the fix is usually retention rather than a bigger machine.



```
airflow db clean --clean-before-timestamp 2024-01-01  
airflow db clean --clean-before-timestamp 2024-01-01 --tables task_instance,dag_run,log,
```

Raising `min_file_process_interval` is the cheapest parsing win. The default re-parses every file every thirty seconds. Sixty or a hundred and twenty seconds is almost always acceptable. New DAGs appear a minute later, and the scheduler does a fraction of the work.

Retention is not optional at scale

Without `db clean` on a schedule, `task_instance` and `log` grow forever. A multi-hundred-gigabyte metadata database makes the UI unusable, slows the scheduler loop, and turns every upgrade migration into a multi-hour outage. Set retention before you need it. Retrofitting it onto a huge table means a long, careful, out-of-hours operation.

► TRY IT

1. Run `airflow dags report` and identify your slowest-parsing file.
2. Find your metadata database size, and the row counts of `task_instance`, `log`, and `xcom`.
3. Raise `min_file_process_interval` to 120 and observe the scheduler's CPU.
4. Run `airflow db clean --dry-run` for a cutoff a year ago and read how many rows it would remove.

Expected result – your own parse-time distribution, real table row counts, and a measurable CPU drop from one configuration line. That `db clean --dry-run` number is usually large enough to schedule the real thing immediately.

Cost

Airflow's cost is rarely the scheduler. It is idle capacity, the database, and the compute your tasks trigger.



Celery workers	Provisioned capacity x time	Sized for the zain peak, idle for twenty hours
Kubernetes pods	Pod-seconds	Thousands of tiny pods, each with fixed startup overhead
Metadata database	Instance size + storage	An unretained history nobody queries
Triggerer	One or two small processes	Almost nothing. This is the point
Task compute	Whatever your tasks call	Sensors in <code>poke</code> mode holding expensive workers
Logs	Object storage + requests	Verbose logs kept forever

Five levers, roughly in order of return:

01

Deferrable operators everywhere

A sensor in `poke` mode on a GPU-capable worker is the most expensive way to wait ever invented. Deferring moves the wait to one small async process.

02

Right-size the executor to the workload

Many short tasks favour Celery, because pod startup dominates. Few heavy tasks favour Kubernetes, because idle workers dominate. A `CeLeryKubernetesExecutor` with queue routing gets both.

03

Scale workers on queue depth

KEDA on Celery queue length, or Kubernetes with scale-to-zero. Sizing for peak and paying overnight is the default and the most common waste.

04

Retention on everything

`db clean` for the database, a lifecycle rule for remote logs. Both grow silently and neither is ever read after a month.



rows, and possibly pods, for twenty seconds of real work.

KEDA scaler for Celery workers

Copy

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
spec:
  scaleTargetRef:
    name: airflow-worker
  minReplicaCount: 1
  maxReplicaCount: 20
  triggers:
    - type: postgresql
      metadata:
        query: >-
          SELECT ceil(count(*)::decimal / 16)
          FROM task_instance
          WHERE state = 'queued' OR (state = 'running' AND queue = 'default')
        targetQueryValue: "1"
```

Publish the number or it will not be managed

Worker utilisation, task-seconds divided by worker-seconds paid for, is the single most revealing Airflow cost metric, and it is usually far lower than people guess. Put it on a dashboard next to the monthly figure. A team that can see 12% utilisation will fix it; a team that cannot see it will keep adding workers.

► TRY IT

1. Calculate worker utilisation for last week: total task duration divided by total worker uptime.
2. Count your sensors and check how many are still in `poke` mode.
3. Look at your peak-to-trough task concurrency over a day. The ratio is your autoscaling opportunity.
4. Estimate your metadata database's growth rate and how much of it is older than ninety days.

Expected result – a utilisation figure that is usually embarrassing, a list of sensors to convert, and a clear autoscaling case. That peak-to-trough ratio in



Observability and SLOs

Mid added alerting on failures. At platform scale you need to know Airflow is healthy **before** a pipeline misses, which means metrics rather than alerts.

airflow.cfg

Copy

```
[metrics]
statsd_on = True
statsd_host = statsd.monitoring
statsd_port = 8125
statsd_prefix = airflow
```

The metrics that predict incidents:

Metric	Watch for	Means
<code>scheduler.scheduler_loop_duration</code>	Rising above a few seconds	The scheduler is falling behind
<code>scheduler.tasks.starving</code>	Non-zero and sustained	Tasks want to run and cannot: pool or slot exhaustion
<code>dagbag_import_errors</code>	Any non-zero value	A broken DAG file is deployed right now
<code>dag_processing.total_parse_time</code>	Rising trend	Top-level code creeping in
<code>executor.queued_tasks</code>	Growing without draining	Worker capacity or queue routing problem
<code>executor.open_slots</code>	At zero for long periods	Saturated executor



<code>dagrun.duration.success. <dag_id></code>	p95 rising	A pipeline degrading before it fails
<code>dagrun.schedule_delay. <dag_id></code>	Rising	Runs starting later than they should
<code>sla_missed</code>	Any	A consumer-facing promise broken

Then define SLOs on the things consumers care about:

SLO	Example target	Why it is the right shape
Freshness	"The events table is current as of 06:00, 99% of days"	What consumers experience
Completion	"The pipeline completes within 90 minutes, p95"	Detects degradation before failure
Scheduling punctuality	"Runs start within 5 minutes of schedule, 99.9%"	Isolates platform health from DAG health
Platform availability	"The scheduler heartbeats, 99.9%"	The thing the platform team owns

That last distinction is the one that makes a platform team's life livable: **separate platform SLOs from pipeline SLOs**. "Runs start on time" is the platform's promise. "The data is correct and complete" is the DAG owner's. Conflating them means the platform team is paged for every team's data bug.

The one alert every Airflow deployment needs

Scheduler heartbeat age. If the scheduler stops, everything stops: no failures, no alerts, no red DAGs, just silence, because failure alerts come from tasks that never ran. A heartbeat alert is the only thing that catches it, and its absence is why "we found out at 10am that nothing had run since 2am" is such a common story.



1. Find whether metrics are exported at all in your deployment. If not, that is your first task.
2. Chart scheduler loop duration and total parse time for the last week and look for a trend.
3. Check `tasks.starving`. A non-zero value you did not know about is common.
4. Stop the scheduler for five minutes and confirm something alerts. If nothing does, add it today.

Expected result – a trend line for the two metrics that predict scheduler trouble, and, in step four, the discovery that most deployments are silent when the scheduler dies. That fourth test is worth running in production during a quiet window.

Upgrades and migrations

An Airflow upgrade touches the database schema, the provider packages, and every DAG's imports at once. Doing it without a plan is how teams end up three major versions behind.

01 Read the release notes properly

Deprecations become removals a version later. The upgrade that breaks you is usually one you skipped, not the one you are doing.

02 Run the upgrade check

`airflow db check-migrations` and, on older versions, the dedicated upgrade-check tooling. Fix what it reports before touching anything.

03 Pin providers separately from core

Provider packages version independently. Upgrading a provider for a bug fix should not require an Airflow upgrade, and that is only true if you pin them separately.

04 Rehearse the migration on a copy of production data

`airflow db migrate` on a two-hundred-gigabyte `task_instance` table can take hours. Time it on a restored snapshot, not in the maintenance window.



a short one.

06

Deploy DAG compatibility ahead of the upgrade

Import paths change between versions. Where both old and new imports work, move to the new ones in advance so the upgrade itself is only infrastructure.

BASH

Copy

```
# Before
airflow db check-migrations
airflow db clean --clean-before-timestamp $(date -d '90 days ago' -I)
pg_dump airflow > backup-$(date -I).sql          # non-negotiable

# The upgrade
pip install "apache-airflow==2.10.*" --constraint "$CONSTRAINTS_URL"
airflow db migrate

# After
airflow dags list-import-errors
airflow jobs check --job-type SchedulerJob --hostname "$(hostname)"
```

requirements.txt

Copy

```
apache-airflow==2.10.2
apache-airflow-providers-amazon==8.28.0
apache-airflow-providers-postgres==5.12.0
# Always install with the official constraints file for your version and Python
```

There is no downgrade path for the database

`airflow db migrate` applies schema changes that are not reversible in practice. Your rollback plan is a database backup taken immediately before, and it must be tested: a backup you have never restored is a hypothesis. Take it, verify you can restore it into a scratch instance, `then` migrate.



1. Find your current Airflow and provider versions, and how far behind the current release you are.
2. Run `airflow db check-migrations` and read the output.
3. Restore a production snapshot into a scratch instance and time `airflow db migrate` against it.
4. Grep your DAGs for deprecated import paths and fix them ahead of any upgrade.

Expected result – a real migration duration measured rather than guessed, and a list of import changes you can deploy independently. Step three is the difference between a planned twenty-minute window and an unplanned three-hour one.

Deployment and DAG delivery

How DAG files reach the scheduler is an architectural choice with real consequences.

Method	How	Trade-off
Baked into the image	DAGs in the Docker image; deploy = new image	Fully atomic and versioned; requires a rollout per DAG change
Git-sync sidecar	A container pulls a branch on an interval	Fast iteration; a bad commit reaches production in seconds
Shared volume / object storage sync	Files synced to a mounted path	Simple; weak atomicity and provenance
Airflow REST API / DAG bundles	Managed deployment tooling	Depends on your platform

✓ Baked image

- DAGs, providers, and dependencies versioned together
- Rollback is redeploying the previous image
- No partial state during deployment

✗ Git-sync

- A DAG change is live within a minute
- No image build per change
- But: a broken DAG is deployed as fast as a good one



Whichever you pick, the gate is the same:

.github/workflows/dags.yml

Copy

```
name: DAGs
on: [pull_request]
jobs:
  gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11', cache: pip }
      - run: pip install -r requirements.txt --constraint constraints.txt
      - run: ruff check dags/ plugins/
      - name: Integrity – no import errors, and standards enforced
        run: pytest tests/test_dag_integrity.py -v
      - name: Unit tests for extracted logic
        run: pytest tests/unit -v
      - name: Parse-time budget
        run: python scripts/assert_parse_time.py --max-seconds 2
```

Git-sync plus a hard CI gate is the pragmatic middle

Baked images are safer; git-sync is faster to iterate with; a required, fast integrity check on the branch that git-sync follows gets you most of both. What is not acceptable is git-sync from a branch anyone can push to without CI. That is a production deployment with no gate.

► TRY IT

1. Identify how DAGs currently reach your scheduler, and how long a change takes to appear.
2. Work out how you would roll back a bad DAG change. Time it.
3. Confirm whether a broken DAG file can reach production without CI passing.
4. Add a parse-time budget assertion to CI and watch it fail on a deliberately slow DAG.



parse time gate is the cheapest way to stop scheduler degradation creeping in.

Running Airflow as a platform

Once several teams share one Airflow, the highest-leverage work stops being technical and becomes about defaults. Four decisions cover most of it.

Publish a template repository, not documentation. A working DAG with retries, timeouts, tags, `doc_md`, and idempotent SQL; the integrity test suite; the CI workflow. Teams inherit every good decision by cloning, and a wiki page is read once.

Publish a shared operator library as a versioned package. The row-count check, the Slack alert callback, the standard extract pattern. It is where you enforce standards: a quality gate that is one line gets adopted, and forty lines does not.

Own the contract, not the DAGs. The platform team owns scheduler health, punctuality, the shared library, and the deployment gate. DAG owners own correctness, their SLAs, and their own on-call. Writing that split down is what stops the platform team being paged for every data bug.

Measure and publish. Without numbers, "Airflow is fine" is an opinion:

Metric	Why
Scheduling punctuality, p99	The platform's own SLO
Total parse time, and the slowest files	Predicts scheduler degradation
Worker utilisation	Cost, and the autoscaling case
DAGs with no retries, no timeout, or no owner	Standards adoption, per team
Import errors, count and duration	A broken DAG in production right now



Metadata database size and growth	The most common scaling ceiling
Sensors still in <code>poke</code> mode	Available cost and capacity win

The failure mode of a shared Airflow

One team's DAG parses slowly, or floods the workers with a backfill, and every other team's pipelines are late. Without pools, per-team queues, and a parse-time budget enforced in CI, the platform is one careless commit away from an incident that is not the platform's fault but is the platform's problem. On a shared deployment, containing noisy neighbours is the primary engineering requirement.

► TRY IT

1. Build a template repository with a working DAG, the integrity tests, and CI. Use it for one real pipeline.
2. Query your metadata database for DAGs with no retries, no `dagrun_timeout`, or no owner. Count them per team.
3. Pick two of the metrics above and publish them somewhere the teams see.
4. Write the one-page contract: what the platform promises, what DAG owners own, and how fast you answer a request.

Expected result – a template that makes the right thing the default, and a standards-adoption number per team that is usually worse than expected. Step four is what determines whether the platform is trusted or worked around.

Incident playbooks

Under pressure, ordering matters more than knowledge. Five shapes, with the first move for each.



Nothing is running. Nothing is `running`. Confirm first with `airflow jobs check --job-type SchedulerJob`, because a dead scheduler produces silence rather than alerts.

Then work the four causes in order of likelihood: is the process alive; can it reach the metadata database; is one DAG file taking longer than `dag_file_processor_timeout` and blocking a processor slot; and is the database itself refusing connections because the pool is exhausted. `airflow dags report` names a pathological file immediately, and `.airflowignore` gets it out of the way while you fix it. Afterwards: the heartbeat alert, if you did not already have it.

Tasks are stuck in `queued`

They are scheduled but nothing picks them up, which is always a capacity or routing problem. Check in order: `parallelism` cluster-wide, then the DAG's `max_active_tasks`, then pool slots, `airflow pools list` shows used against total, then whether the tasks name a `queue` that no running worker consumes. That last one is the sneaky case, because it looks identical to capacity exhaustion and no amount of extra workers fixes it.

The metadata database is degrading

The UI times out, the scheduler loop lengthens, everything is slow. Check size and row counts first, then the connection count against the server's limit. The immediate action is `db clean` with a conservative cutoff; the durable fixes are scheduled retention, PgBouncer in front of a Postgres with hundreds of Airflow connections, and moving large XCom values out of the database. Resist the instinct to resize the instance first. It buys weeks and hides the cause.

A backfill took production down



0m

Stop the backfill

Pause the DAG. Then clear the queued backfill task instances so they stop competing.



2m

Confirm production recovers

Watch queue depth drain and normal pipelines resume before doing anything else.



10m

Restart it constrained

`max_active_runs=1` , a dedicated pool with few slots, and low `priority_weight` .

after

Close the loop

A standing backfill pool, and a rule that backfills run at low priority outside business hours.

A secret leaked into a task log

Rotate before anything else. The log is in object storage, in a log aggregator, possibly in a Slack alert, and in anyone's browser cache. Deleting the log line is not containment. Then find the code path that printed it, and fix the class rather than the instance: add the key name to `sensitive_var_conn_names` , and add a CI check that greps DAG code for printing connection or variable values. Rotating and moving on without closing the class means it happens again with a different credential.

► TRY IT

1. Stop your scheduler in a test environment and time how long until you find out from monitoring rather than by looking.
2. Create a pool with one slot, queue ten tasks into it, and practise diagnosing "stuck in queued" from the UI alone.
3. Add a task to a DAG that names a non-existent queue and confirm it queues forever with no error.
4. Write your own version of these playbooks for your deployment, and put them where an on-call engineer will find them.

Expected result – a measured detection time for the worst failure mode, and a task queued forever with no error message anywhere. Run the third experiment at least once. It is the hardest failure here to diagnose if you have never seen it.



Naming the boundary is itself a senior signal, and it lands better than defending Airflow for every workload.

✓ **Airflow is the right tool**

- Scheduled batch work with real dependencies
- Heterogeneous steps across many systems
- Backfills and reprocessing of historical intervals
- Pipelines needing retries, alerting, and an audit trail
- Orchestration where the work happens elsewhere

✗ **Reach for something else**

- Sub-second or event-driven latency → Kafka, Flink, a queue
- Pure in-warehouse SQL transforms → dbt, on a schedule
- Per-request or per-event triggering → a service, not a DAG
- Heavy in-process data crunching → Spark, Ray, the warehouse
- Thousands of tiny tasks per minute → batch them, or a queue

Tool	Solves	Overlaps Airflow on
dbt	SQL transformation, testing, and lineage inside the warehouse	Transform DAGs: usually run <i>from</i> Airflow, not instead
Dagster	Asset-centric orchestration with strong typing and lineage	Directly; a genuine alternative with different ergonomics
Prefect	Python-native flows with dynamic control flow	Directly; lighter for dynamic workloads
Step Functions / Cloud Composer	Managed orchestration	Airflow itself, in the managed case
Kafka / Flink	Streaming and event processing	Nothing: a different latency class
Argo Workflows	Kubernetes-native container DAGs	Container orchestration, without Airflow's scheduling model

The shape that works in practice: **Airflow schedules and orchestrates, the heavy work happens in a purpose-built engine, and transformation logic lives in dbt or SQL rather than**



worker memory is a misuse that will eventually force a rewrite.

The honest answer about "should we move off Airflow?"

Usually the problem is not Airflow but how it is being used: top-level code, sensors in `poke` mode, data processing inside tasks, no retention, no pools. Those are all fixable in weeks. Migrating orchestrators is a multi-quarter project that reproduces every one of those mistakes in a new tool unless the underlying practices change. Diagnose the practices before proposing the migration.

► TRY IT

1. List your heaviest DAGs and classify each task: does the work happen in the worker, or elsewhere?
2. For anything doing work in the worker, decide where it should happen instead.
3. Write down the specific requirement that would justify a different orchestrator.
4. Name the one thing in your stack currently orchestrated by nothing: a cron job, a notebook, a person.

Expected result – a clear split between orchestration and computation, and usually one unorchestrated component nobody had noticed. That last answer is often the most valuable thing on this page.

The review checklist

Take this checklist away from this level. Run it against any DAG (your own, or one you are reviewing) and it catches nearly everything covered here before it becomes an incident.

Check	Looking for	Level
Is <code>catchup=False</code> unless a backfill is intended?	No accidental flood of runs	Beginner



Does the DAG use <code>task.now()</code> ?	Retries and backfills are reproducible	Beginner
Are tasks idempotent: delete the partition, then write?	Retries and backfills are safe	Beginner
Are <code>retries</code> and <code>retry_delay</code> set?	Transient failures self-heal	Beginner
Is <code>execution_timeout</code> set on every task?	A hung task cannot hold a slot forever	Beginner
Is <code>dagrun_timeout</code> set?	A stuck run cannot pile up intervals	Beginner
Is <code>max_active_runs</code> set on anything that writes?	No concurrent writers to one table	Beginner
Is XCom carrying metadata only?	The metadata database stays healthy	Beginner
Are credentials in a Connection, never in code?	No secret in the repository	Beginner
Is there no top-level query, API call, or heavy import?	The scheduler stays fast	Beginner
Are sensors deferrable, or at least <code>reschedule</code> with a timeout?	No worker slots wasted waiting	Mid
Do joins after a branch have an explicit trigger rule?	No silent skip cascade	Beginner
Are shared external systems protected by a pool?	One DAG cannot exhaust a database	Mid
Is there an <code>on_failure_callback</code> with a log link?	Alerts are actionable	Mid
Does the DAG have <code>tags</code> , an owner, and <code>doc_md</code> ?	Ownership and a runbook exist	Mid



Does the integrity test suite pass in CI?	A broken DAG cannot ship	Mid
Is the parse time within budget?	No scheduler degradation	Mid
Are secrets in a secrets backend, not the metadata DB?	Rotation, scoping, and an audit trail	Senior
Is per-DAG access control set for multi-team deployments?	Teams cannot trigger each other's pipelines	Senior
Is <code>db clean</code> scheduled?	The database does not become the ceiling	Senior
Is scheduler heartbeat alerting in place?	Silence is detected	Senior
Do backfills run in a constrained pool at low priority?	Production is not collateral damage	Senior
Does the heavy work happen outside the worker?	Airflow orchestrates rather than computes	Senior

Automate the mechanical half

Most of the Beginner and Mid rows are machine-checkable from the `DagBag` : retries, timeouts, tags, owner, `doc_md` , `catchup` , and parse time. Put them in the integrity test and reserve human review for the judgement calls: is this task idempotent, is that pool size right, should this work be in a worker at all.

The complete picture

The series' final artefact: every level's topics, hardened. Nothing in it is new.

```
days/events_platform.py
```

Copy



```
"""owner:"data-platform" * consumers.* exec dashboard, ML features
**SLO:** green by 07:00 UTC, 99% of days.
**On failure:** idempotent – safe to clear and rerun any interval.
**Escalate:** #data-oncall if not green by 07:30 UTC.
"""

from datetime import datetime, timedelta

from airflow.datasets import Dataset
from airflow.decorators import dag, task, task_group
from airflow.models.param import Param
from airflow.operators.empty import EmptyOperator
from airflow.providers.amazon.aws.hooks.s3 import S3Hook
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
from airflow.timetables.trigger import CronTriggerTimetable
from airflow.utils.trigger_rule import TriggerRule

from company_airflow.callbacks import slack_alert # the shared library
from company_airflow.operators import RowCountCheckOperator

EVENTS = Dataset("postgres://warehouse/analytics.daily_events")

default_args = {
    "owner": "data-platform",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "max_retry_delay": timedelta(minutes=30),
    "execution_timeout": timedelta(minutes=30), # no task holds a slot forever
    "on_failure_callback": slack_alert, # actionable, with a log link
    "sla": timedelta(hours=2),
}

@dag(
    dag_id="events_platform",
    schedule=CronTriggerTimetable("0 2 * * *", timezone="Europe/Berlin"),
    start_date=datetime(2024, 1, 1),
    catchup=False, # backfill deliberately
    default_args=default_args,
    dagrun_timeout=timedelta(hours=4),
    max_active_runs=1, # one writer per table
    max_active_tasks=12, # do not starve other teams
    tags=["production", "events", "tier-1"],
    doc_md=__doc__,
```



```
        "Viewer": {"can_read"},
    },
    params={
        "full_refresh": Param(False, type="boolean", title="Full refresh"),
        "batch_size": Param(5000, type="integer", minimum=100, maximum=50_000),
    },
    on_failure_callback=slack_alert,
)

def events_platform():

    start = EmptyOperator(task_id="start")

    # Deferrable: zero worker slots held for up to three hours of waiting.
    wait = S3KeySensor(
        task_id="wait_for_landing",
        bucket_name="raw-data",
        bucket_key="events/{{ ds }}/_SUCCESS",
        aws_conn_id="aws_raw",          # resolved from the secrets backend
        deferrable=True,
        poke_interval=60,
        timeout=60 * 60 * 3,
    )

    @task
    def list_partitions() → list[str]:
        """Run-time listing. Nothing at module level touches the network."""
        return S3Hook(aws_conn_id="aws_raw").list_keys(
            bucket_name="raw-data", prefix="events/{{ ds }}/part-"
        )

    @task(pool="warehouse_writes", pool_slots=1, priority_weight=10)
    def load_partition(key: str) → int:
        """Pooled: the warehouse's connection limit is respected cluster-wide.
        Each partition retries independently."""
        return copy_into_staging(key)

    @task_group(group_id="quality")
    def quality(counts: list[int]):
        # A data problem will not fix itself: this raises AirflowFailException.
        RowCountCheckOperator(
            task_id="row_count",
            conn_id="warehouse",
            table="staging.events",
            day="{{ ds }}",
```



```
@task
def totals(c: list[int]) → int:
    return sum(c)

totals(counts)

# Idempotent: DELETE the partition then INSERT. Publishes the dataset.
rollup = SQLExecuteQueryOperator(
    task_id="rollup",
    conn_id="warehouse",
    sql="sql/daily_rollup.sql",
    params={"lookback_days": 7},
    outlets=[EVENTS],          # downstream DAGs are data-scheduled
)

cleanup = EmptyOperator(task_id="cleanup", trigger_rule=TriggerRule.ALL_DONE)
end = EmptyOperator(task_id="end")

counts = load_partition.expand(key=list_partitions())
start >> wait >> counts
quality(counts) >> rollup >> cleanup >> end

events_platform()
```

airflow.cfg: the platform side

Copy

```
[core]
executor = CeleryKubernetesExecutor
parallelism = 128
max_active_tasks_per_dag = 16
hide_sensitive_var_conn_fields = True

[secrets]
backend = airflow.providers.hashicorp.secrets.vault.VaultBackend

[scheduler]
parsing_processes = 4
min_file_process_interval = 120          # cheapest parsing win there is
use_row_level_locking = True             # multiple active schedulers

[metrics]
statsd_on = True                         # loop duration, starving tasks, parse time
```



```
expose_config = False
```

BASH

Copy

```
# Verify, do not assume
airflow jobs check --job-type SchedulerJob      # is the scheduler alive?
airflow dags report                             # parse-time distribution
airflow dags list-import-errors                 # anything broken right now?
airflow pools list                             # used vs total slots
airflow db clean --dry-run --clean-before-timestamp $(date -d '90 days ago' -I)
```

► TRY IT: THE FINAL EXERCISE

1. Build this end to end on a real pipeline: a secrets backend, a deferrable sensor, pooled writes, dataset publication, per-DAG access control, and the shared library.
2. Verify each control actively rather than trusting it: confirm the deferred sensor holds no worker slot; confirm the pool caps concurrency regardless of cluster capacity; confirm a low-privilege user cannot trigger it; confirm deleting the connection from the UI does not break it.
3. Reproduce a run from three months ago by clearing that interval, and confirm the output is identical rather than duplicated.
4. Then hand it to a colleague and ask them to run the review checklist against it.

Expected result – several refusals and one successful historical rerun. A control you have never watched refuse anything is decoration. Step two is the difference between a pipeline that looks governed and one that is. The colleague review will find something you did not, which is the point.

Where the series leaves you

Across the three levels you have gone from a first three-task DAG to owning Airflow as a platform. The same topics carried all the way through, each time with more depth:



You should now be able to look at an Airflow deployment and see not just what it does but what it **guarantees**: whether a run from last quarter can be reproduced, whether a hung task can take the cluster with it, who can read which credential, which component will saturate first and what the metric for it is, what it costs and how much of that is idle, whether a broken DAG can reach production, and whether anyone finds out when the scheduler stops.

Can you...	
Say why DAG-folder write access is administrative access?	A DAG file is arbitrary Python on your workers
Name what RBAC does <code>not</code> isolate?	Task code and credentials between teams
Give the honest answer on multi-tenancy?	Hostile tenants need separate deployments
Say what a secrets backend buys you?	Rotation, per-team scoping, an audit trail
Name the four scaling bottlenecks and their metrics?	Parsing, scheduler loop, database, workers
Give the cheapest parsing win?	Raise <code>min_file_process_interval</code>
Say what is almost always the real ceiling?	The metadata database: retention, not size
Name the biggest cost lever?	Deferrable operators, then autoscaling on queue depth
Give the one alert every deployment needs?	Scheduler heartbeat age
Say what the rollback plan for <code>db migrate</code> is?	A tested backup. There is no downgrade
Explain the split between platform and pipeline SLOs?	Punctuality is platform; correctness is the owner
Name the first move when nothing is running?	<code>airflow jobs check</code> : silence, not failure
Say what a task queued forever with no error means?	Often a <code>queue</code> no worker consumes



Say where Airflow stops:	streaming, sub-second latency, in-worker compute
Give the honest answer to "should we migrate off"?	Usually the practices, not the tool

That review instinct matters more than any single configuration flag, because, as with containers and data versioning, most orchestration incidents are prevented at review time rather than at 3am.



PREVIOUS

Mid-level · Tips & Tricks

NEXT UP

Senior · Interview Review



Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

Roadmaps

Student Guides

Courses



MLOps MENA
COMMUNITY

العربية العربية



Join the Community ↗



Articles

COMMUNITY

Meet the team

Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**